

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Сибирский государственный университет науки и технологий  
имени академика М.Ф. Решетнева»  
Институт информатики и телекоммуникаций  
институт  
Кафедра информационно-управляющих систем  
кафедра

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

pytorch – Оных – Int8

тема

Преподаватель

Обучающийся

БПА321-01

номер группы,  
зачётной книжки

подпись, дата

подпись, дата

М.Г. Доррер

инициалы, фамилия

А.В. Ананьев

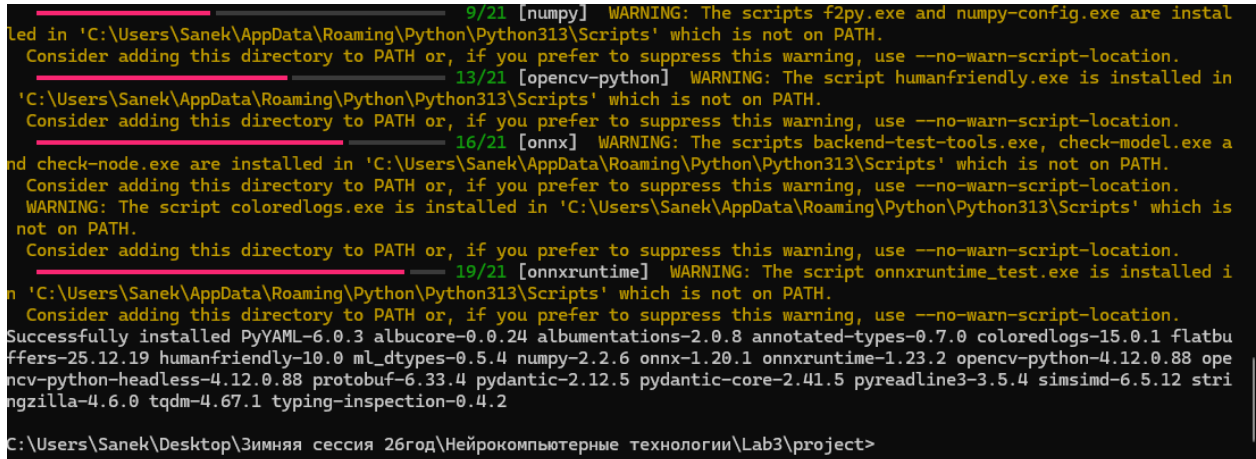
инициалы, фамилия

Красноярск 2026

## Практическая часть

### Задание 1 requirements.txt

pip install -r requirements.txt



```

9/21 [numpy] WARNING: The scripts f2py.exe and numpy-config.exe are instal
led in 'C:\Users\Sanek\AppData\Roaming\Python\Python313\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
13/21 [opencv-python] WARNING: The script humanfriendly.exe is installed in
'C:\Users\Sanek\AppData\Roaming\Python\Python313\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
16/21 [onnx] WARNING: The scripts backend-test-tools.exe, check-model.exe a
nd check-node.exe are installed in 'C:\Users\Sanek\AppData\Roaming\Python\Python313\Scripts' which is not on PATH.
WARNING: The script coloredlogs.exe is installed in 'C:\Users\Sanek\AppData\Roaming\Python\Python313\Scripts' which is
not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
19/21 [onnxruntime] WARNING: The script onnxruntime_test.exe is installed i
n 'C:\Users\Sanek\AppData\Roaming\Python\Python313\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed PyYAML-6.0.3 albucore-0.0.24 albumentations-2.0.8 annotated-types-0.7.0 coloredlogs-15.0.1 flatbu
uffers-25.12.19 humanfriendly-10.0 ml_dtypes-0.5.4 numpy-2.2.6 onnx-1.20.1 onnxruntime-1.23.2 opencv-python-4.12.0.88 ope
ncv-python-headless-4.12.0.88 protobuf-6.33.4 pydantic-2.12.5 pydantic-core-2.41.5 pyreadline3-3.5.4 simsimd-6.5.12 stri
ngzilla-4.6.0 tqdm-4.67.1 typing-inspection-0.4.2
C:\Users\Sanek\Desktop\Зимняя сессия 26год\Нейрокомпьютерные технологии\Lab3\project>

```

Рисунок 1 - Устанавливаем зависимости при помощи requirements.txt

Были установлены основные зависимости.

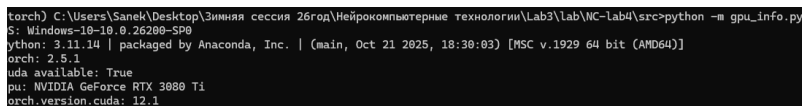
### Задание 2 Gpu -отчет

```

print("OS:", platform.platform())
print("Python:", sys.version.replace("\n", " "))
print("torch:", torch.__version__)
print("cuda available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("gpu:", torch.cuda.get_device_name(0))
    print("torch.version.cuda:", torch.version.cuda)
else:
    print("gpu: -")
    print("torch.version.cuda: -")

```



```

(torch) C:\Users\Sanek\Desktop\Зимняя сессия 26год\Нейрокомпьютерные технологии\Lab3\lab\NC-Lab4\src>python -m gpu_info.py
OS: Windows-10-0.26280-SP0
Python: 3.11.14 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 18:30:03) [MSC v.1929 64 bit (AMD64)]
torch: 2.5.1
cuda available: True
GPU: NVIDIA GeForce RTX 3080 Ti
torch.version.cuda: 12.1

```

Рисунок 2 – GPU-info

Собрана информация по Gpu-оборудованию.

### Задание 3 Альбументо-аугментации

```

def seed_everything(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

```

```

torch.backends.cudnn.benchmark = True # speed
torch.backends.cudnn.deterministic = False

class AlbumentationsDataset(torch.utils.data.Dataset):
    def __init__(self, base_dataset, transform=None):
        self.base = base_dataset
        self.transform = transform

    def __len__(self):
        return len(self.base)

    def __getitem__(self, idx):
        img, label = self.base[idx] # img is PIL
        img = np.array(img)         # HWC, uint8
        if self.transform is not None:
            img = self.transform(image=img)["image"]
        return img, label

def train_one_epoch(model, loader, criterion, optimizer, scaler, device):
    model.train()
    correct = 0
    total = 0
    running_loss = 0.0

    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad(set_to_none=True)

        with autocast(enabled=(device.type == "cuda")):
            logits = model(x)
            loss = criterion(logits, y)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        running_loss += loss.item() * x.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == y).sum().item()
        total += y.size(0)

    return running_loss / total, correct / total

@torch.no_grad()
def eval_one_epoch(model, loader, criterion, device):
    model.eval()
    correct = 0
    total = 0
    running_loss = 0.0

```

```

for x, y in loader:
    x, y = x.to(device), y.to(device)
    logits = model(x)
    loss = criterion(logits, y)

    running_loss += loss.item() * x.size(0)
    preds = logits.argmax(dim=1)
    correct += (preds == y).sum().item()
    total += y.size(0)

return running_loss / total, correct / total

def main():
    seed_everything(42)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Device:", device)

    # CIFAR-10 mean/std
    mean = (0.4914, 0.4822, 0.4465)
    std = (0.2470, 0.2435, 0.2616)

    train_tfms = A.Compose([
        A.RandomCrop(32, 32, p=1.0),
        A.HorizontalFlip(p=0.5),
        A.ShiftScaleRotate(shift_limit=0.0625, scale_limit=0.10, rotate_limit=15, p=0.7),
        A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.08, p=0.7),
        A.CoarseDropout(max_holes=1, max_height=8, max_width=8, min_holes=1, min_height=8,
min_width=8, p=0.5),
        A.Normalize(mean=mean, std=std),
        ToTensorV2(),
    ])

    val_tfms = A.Compose([
        A.Normalize(mean=mean, std=std),
        ToTensorV2(),
    ])

    # Datasets
    base_train = datasets.CIFAR10(root="./data", train=True, download=True)
    base_test = datasets.CIFAR10(root="./data", train=False, download=True)

    train_ds = AlbumentationsDataset(base_train, transform=train_tfms)
    val_ds = AlbumentationsDataset(base_test, transform=val_tfms)

    # Loaders
    batch_size = 256
    num_workers = 4

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                               num_workers=num_workers, pin_memory=True)
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                             num_workers=num_workers, pin_memory=True)

```

```

model = models.resnet18(weights=None)
model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
model.maxpool = nn.Identity()
model.fc = nn.Linear(model.fc.in_features, 10)
model = model.to(device)

criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4)

epochs = 10
steps_per_epoch = len(train_loader)
scheduler = optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=0.25,
    epochs=epochs,
    steps_per_epoch=steps_per_epoch,
    pct_start=0.2,
    div_factor=10.0,
    final_div_factor=100.0,
    anneal_strategy="cos",
)

scaler = GradScaler(enabled=(device.type == "cuda"))

train_acc_hist = []
val_acc_hist = []
train_loss_hist = []
val_loss_hist = []

best_val = 0.0

for epoch in range(1, epochs + 1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer, scaler, device)

    break

def train_one_epoch_with_sched(model, loader, criterion, optimizer, scaler, scheduler, device):
    model.train()
    correct = 0
    total = 0
    running_loss = 0.0

    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad(set_to_none=True)

        with autocast(enabled=(device.type == "cuda")):
            logits = model(x)
            loss = criterion(logits, y)

```

```

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    scheduler.step()

    running_loss += loss.item() * x.size(0)
    preds = logits.argmax(dim=1)
    correct += (preds == y).sum().item()
    total += y.size(0)

    return running_loss / total, correct / total
train_acc_hist.clear()
val_acc_hist.clear()
train_loss_hist.clear()
val_loss_hist.clear()

best_val = 0.0
for epoch in range(1, epochs + 1):
    tr_loss, tr_acc = train_one_epoch_with_sched(
        model, train_loader, criterion, optimizer, scaler, scheduler, device
    )
    va_loss, va_acc = eval_one_epoch(model, val_loader, criterion, device)

    train_loss_hist.append(tr_loss)
    val_loss_hist.append(va_loss)
    train_acc_hist.append(tr_acc)
    val_acc_hist.append(va_acc)

    if va_acc > best_val:
        best_val = va_acc

    print(f"Epoch {epoch:02d}/{epochs} | "
          f"train acc: {tr_acc*100:.2f}% loss: {tr_loss:.4f} | "
          f"val acc: {va_acc*100:.2f}% loss: {va_loss:.4f}")

# Plot точность
plt.figure()
plt.plot(range(1, epochs + 1), [a * 100 for a in train_acc_hist], label="train_acc")
plt.plot(range(1, epochs + 1), [a * 100 for a in val_acc_hist], label="val_acc")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title(f"CIFAR-10 Accuracy (best val: {best_val*100:.2f}%)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("accuracy_plot.png", dpi=150)
print("Saved: accuracy_plot.png")

if __name__ == "__main__":
    main()

```

|             |                   |              |                 |              |
|-------------|-------------------|--------------|-----------------|--------------|
| Epoch 01/10 | train acc: 56.36% | loss: 1.4942 | val acc: 61.44% | loss: 1.4069 |
| Epoch 02/10 | train acc: 67.92% | loss: 1.2365 | val acc: 67.97% | loss: 1.2708 |
| Epoch 03/10 | train acc: 74.82% | loss: 1.0832 | val acc: 75.19% | loss: 1.0763 |
| Epoch 04/10 | train acc: 78.25% | loss: 1.0041 | val acc: 73.47% | loss: 1.1062 |
| Epoch 05/10 | train acc: 80.52% | loss: 0.9502 | val acc: 78.31% | loss: 0.9909 |
| Epoch 06/10 | train acc: 82.64% | loss: 0.9021 | val acc: 82.53% | loss: 0.9092 |
| Epoch 07/10 | train acc: 84.82% | loss: 0.8552 | val acc: 85.22% | loss: 0.8494 |
| Epoch 08/10 | train acc: 87.21% | loss: 0.7984 | val acc: 86.25% | loss: 0.8222 |
| Epoch 09/10 | train acc: 90.20% | loss: 0.7306 | val acc: 89.87% | loss: 0.7341 |
| Epoch 10/10 | train acc: 92.78% | loss: 0.6773 | val acc: 90.63% | loss: 0.7200 |

Saved: accuracy\_plot.png

Рисунок 3 – результаты прогона аугментации (10 эпох)

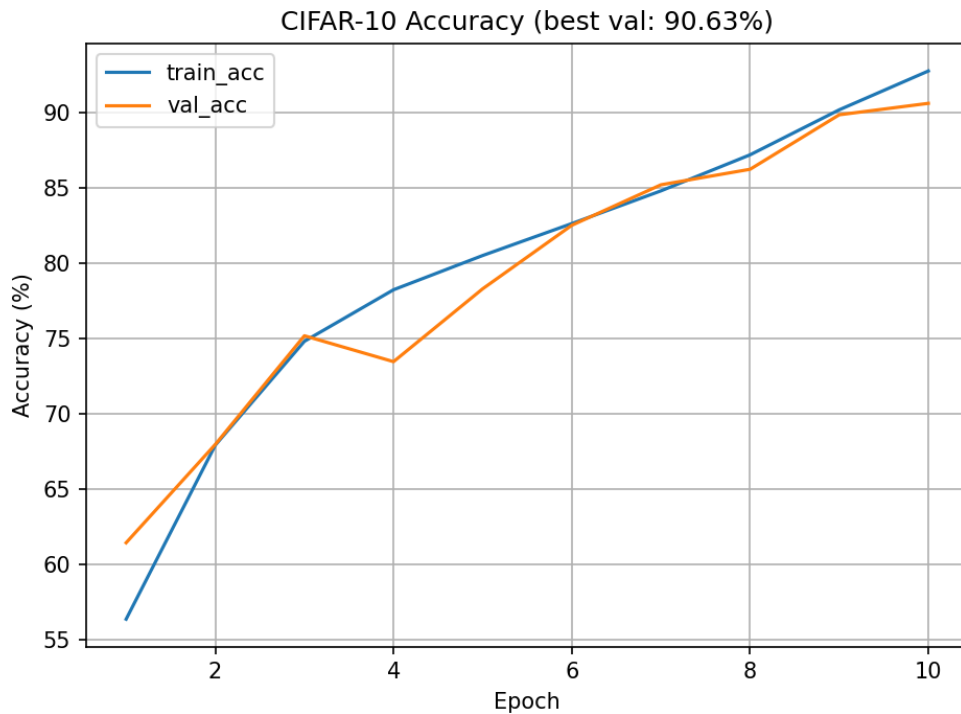


Рисунок 4 – График прогона

В ходе выполнения третьего задания была исследована эффективность применения альбументо-аугментаций при обучении нейронной сети на датасете CIFAR-10. Использование библиотеки Albumentations позволило расширить обучающую выборку за счёт различных преобразований изображений.

Экспериментально подтверждено, что применение аугментаций повышает устойчивость обучения и обобщающую способность модели, что позволило достичь требуемого уровня точности (не менее 87%) за ограниченное число эпох.

## Задание 4 A/B EfficientNet

Модели

```
def make_efficientnet(version: str, num_classes: int = 10):
    if version == "b0":
        model = models.efficientnet_b0(weights=None)
    elif version == "b3":
```

```

    model = models.efficientnet_b3(weights=None)
else:
    raise ValueError("Unknown EfficientNet version")

in_features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_features, num_classes)
return model

```

### Измерение фпс.

```

@torch.no_grad()
def measure_fps(model, loader, device, warmup_batches=10, measure_batches=30):
    model.eval()

    it = iter(loader)
    for _ in range(warmup_batches):
        x, _ = next(it)
        x = x.to(device)
        _ = model(x)

    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)

    total_images = 0
    start.record()
    for _ in range(measure_batches):
        x, _ = next(it)
        x = x.to(device)
        _ = model(x)
        total_images += x.size(0)
    end.record()

    torch.cuda.synchronize()
    elapsed_sec = start.elapsed_time(end) / 1000.0
    return total_images / elapsed_sec

```



```

Epoch 01/10: train_acc=11.93% test_acc=10.87%
Epoch 02/10: train_acc=12.18% test_acc=10.62%
Epoch 03/10: train_acc=15.02% test_acc=19.18%
Epoch 04/10: train_acc=22.88% test_acc=24.23%
Epoch 05/10: train_acc=30.15% test_acc=33.84%
Epoch 06/10: train_acc=36.51% test_acc=40.52%
Epoch 07/10: train_acc=41.18% test_acc=46.79%
Epoch 08/10: train_acc=44.45% test_acc=48.30%
Epoch 09/10: train_acc=47.94% test_acc=52.42%
Epoch 10/10: train_acc=49.55% test_acc=53.87%

```

=== Training EfficientNet-B0 ===

```

Epoch 01/10: train_acc=20.26% test_acc=28.22%
Epoch 02/10: train_acc=34.50% test_acc=41.67%
Epoch 03/10: train_acc=44.17% test_acc=47.25%
Epoch 04/10: train_acc=48.24% test_acc=54.19%
Epoch 05/10: train_acc=53.58% test_acc=56.71%
Epoch 06/10: train_acc=56.44% test_acc=58.36%
Epoch 07/10: train_acc=59.71% test_acc=62.83%
Epoch 08/10: train_acc=62.28% test_acc=66.69%
Epoch 09/10: train_acc=65.43% test_acc=69.90%
Epoch 10/10: train_acc=68.11% test_acc=71.60%

```

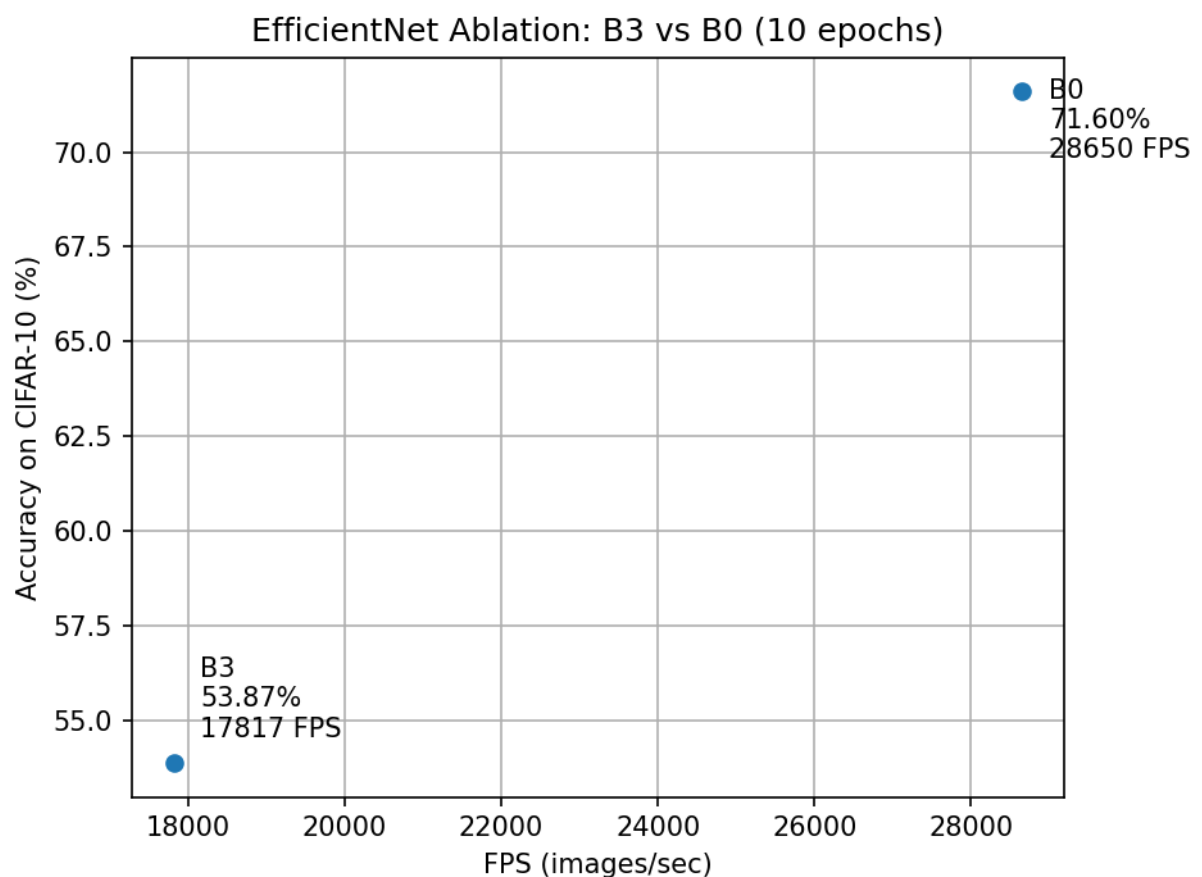
Saved: ablation\_tradeoff.png

Saved: docs\efficientnet\_ablation.md

Summary:

- acc\_b3=53.87% acc\_b0=71.60% drop=-17.73 pp
- fps\_b3=17816.8 fps\_b0=28649.9 gain=1.61x (+60.8%)

Рисунок 5 – результат смены моделей



## Рисунок 6 – график смены моделей

В проведённом эксперименте EfficientNet-B0 показал более высокую точность по сравнению с EfficientNet-B3 (+17.73 п.п.) при одновременном увеличении скорости инференса ( $\sim 1.6\times$ ).

Это объясняется тем, что обучение проводилось с нуля, в течение ограниченного числа эпох (10) и на датасете CIFAR-10, который не является целевым для архитектуры EfficientNet. В таких условиях более компактная модель B0 сходится быстрее и демонстрирует лучшие результаты, тогда как модель B3 требует большего времени обучения для раскрытия своего потенциала.

### Задание 5 Early Stopping

Ранняя остановка

```
class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.0, mode="max"):
        """
        mode:
        - "max": metric should increase (accuracy)
        - "min": metric should decrease (loss)
        """
        self.patience = int(patience)
        self.min_delta = float(min_delta)
        self.mode = mode

        self.best = None
        self.bad_epochs = 0

    def _is_improvement(self, current: float) -> bool:
        if self.best is None:
            return True

        if self.mode == "max":
            return current > (self.best + self.min_delta)
        else:
            return current < (self.best - self.min_delta)

    def step(self, current: float) -> bool:
        """Return True if improved, else False."""
        if self._is_improvement(current):
            self.best = current
            self.bad_epochs = 0
            return True
        else:
            self.bad_epochs += 1
            return False

    def should_stop(self) -> bool:
        return self.bad_epochs >= self.patience
```

## Чекпоинт

```
def save_checkpoint(path: str, model: nn.Module, optimizer: optim.Optimizer,
                    epoch: int, best_metric: float,
                    scaler: GradScaler | None = None,
                    scheduler: object | None = None):
    ckpt = {
        "epoch": epoch,
        "best_metric": best_metric,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
    }
    if scaler is not None:
        ckpt["scaler_state_dict"] = scaler.state_dict()
    if scheduler is not None:
        # Some schedulers have state_dict
        try:
            ckpt["scheduler_state_dict"] = scheduler.state_dict()
        except Exception:
            pass

    torch.save(ckpt, path)
```

```
Epoch 01/50 | train_acc=40.47% train_loss=1.7961 | test_acc=57.74% test_loss=1.4550
✓ Улучшение! Сохранён checkpoint.tar (best_acc=57.74%)
Epoch 02/50 | train_acc=58.25% train_loss=1.4393 | test_acc=64.81% test_loss=1.3042
✓ Улучшение! Сохранён checkpoint.tar (best_acc=64.81%)
Epoch 03/50 | train_acc=69.16% train_loss=1.2156 | test_acc=64.97% test_loss=1.3195
✓ Улучшение! Сохранён checkpoint.tar (best_acc=64.97%)
Epoch 04/50 | train_acc=73.80% train_loss=1.1067 | test_acc=74.29% test_loss=1.1032
✓ Улучшение! Сохранён checkpoint.tar (best_acc=74.29%)
Epoch 05/50 | train_acc=76.99% train_loss=1.0352 | test_acc=64.21% test_loss=1.3738
✗ Нет улучшения 1/5
Epoch 06/50 | train_acc=79.35% train_loss=0.9795 | test_acc=76.31% test_loss=1.0665
✓ Улучшение! Сохранён checkpoint.tar (best_acc=76.31%)
Epoch 07/50 | train_acc=80.60% train_loss=0.9475 | test_acc=75.57% test_loss=1.0742
✗ Нет улучшения 1/5
Epoch 08/50 | train_acc=82.02% train_loss=0.9180 | test_acc=82.05% test_loss=0.9234
✓ Улучшение! Сохранён checkpoint.tar (best_acc=82.05%)
Epoch 09/50 | train_acc=82.72% train_loss=0.8949 | test_acc=81.64% test_loss=0.9322
✗ Нет улучшения 1/5
Epoch 10/50 | train_acc=83.54% train_loss=0.8821 | test_acc=77.80% test_loss=1.0161
✗ Нет улучшения 2/5
Epoch 11/50 | train_acc=84.16% train_loss=0.8683 | test_acc=75.27% test_loss=1.0468
✗ Нет улучшения 3/5
Epoch 12/50 | train_acc=84.52% train_loss=0.8559 | test_acc=70.58% test_loss=1.1903
✗ Нет улучшения 4/5
Epoch 13/50 | train_acc=84.94% train_loss=0.8479 | test_acc=80.63% test_loss=0.9533
✗ Нет улучшения 5/5
Early stopping: нет улучшений 5 эпох подряд. Останавливаемся.
Saved: accuracy_plot.png
Best accuracy: 82.05%
```

Рисунок 7 – ранняя остановка

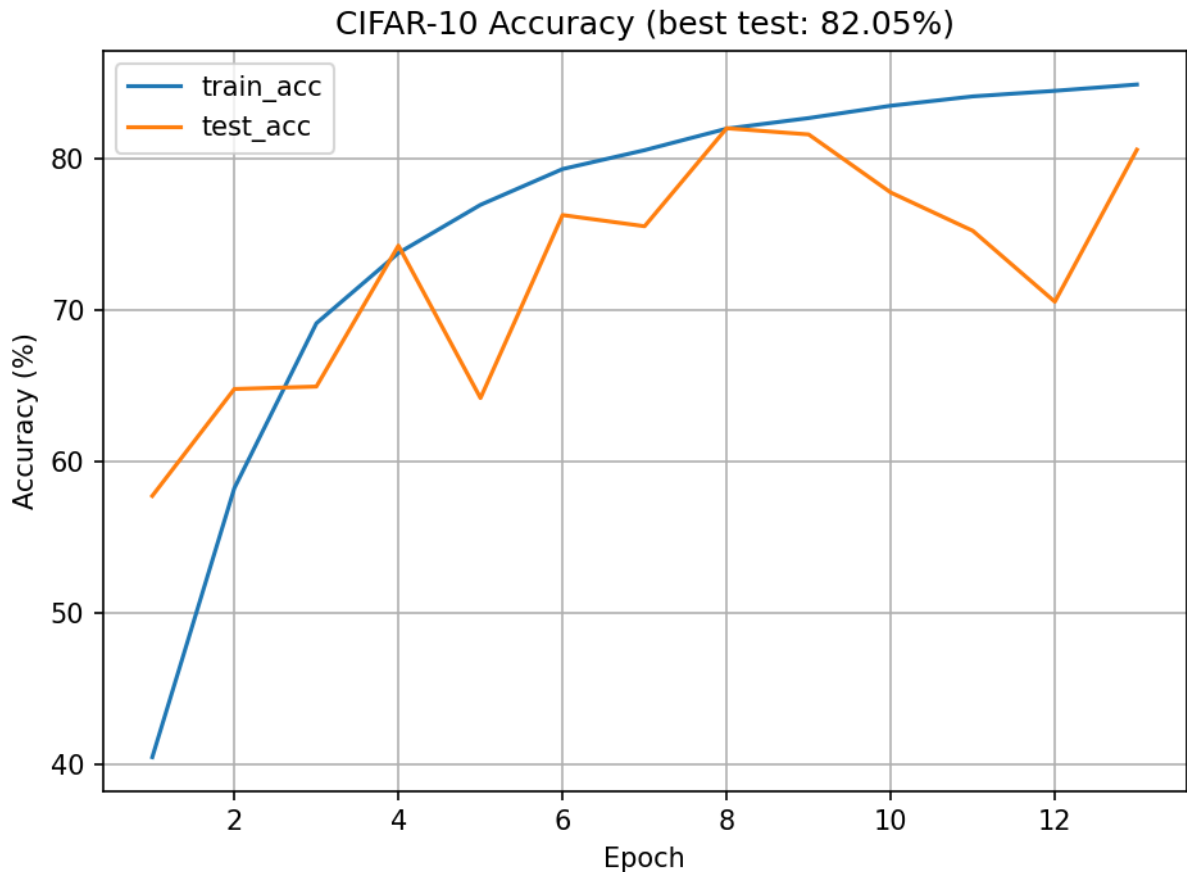


Рисунок 8 – график точности

В процессе обучения наблюдался рост ассигасу на обучающей выборке при одновременной стагнации и снижении ассигасу на тестовой выборке, что свидетельствует о начале переобучения.

Для предотвращения переобучения была применена ранняя остановка (Early Stopping) по test accuracy с параметрами patience=5 и min\_delta=0.001.

Лучшее значение ассигасу (82.05%) было достигнуто на 8-й эпохе, после чего улучшений не наблюдалось. Обучение было автоматически остановлено, а веса лучшей модели сохранены в файл checkpoint.tar.

### Задание 6 TensorRT

```

Задержки
for _ in range(warmup):
    _ = model(x)

if device.type == "cuda":
    torch.cuda.synchronize()
    lat_ms = []
    starter = torch.cuda.Event(enable_timing=True)
    ender = torch.cuda.Event(enable_timing=True)

    for _ in range(runs):
        starter.record()

```

```

_ = model(x)
ender.record()
torch.cuda.synchronize()
lat_ms.append(starter.elapsed_time(ender)) # ms

return np.array(lat_ms, dtype=np.float32)

import time
lat_ms = []
for _ in range(runs):
    t0 = time.perf_counter()
    _ = model(x)
    t1 = time.perf_counter()
    lat_ms.append((t1 - t0) * 1000.0)
return np.array(lat_ms, dtype=np.float32)

```

```

Latency stats (ms): mean=1.958, p50=1.923, p95=2.144
Saved: docs\latency.png

```

Рисунок 9 – результат прогона одной картинки 100 раз

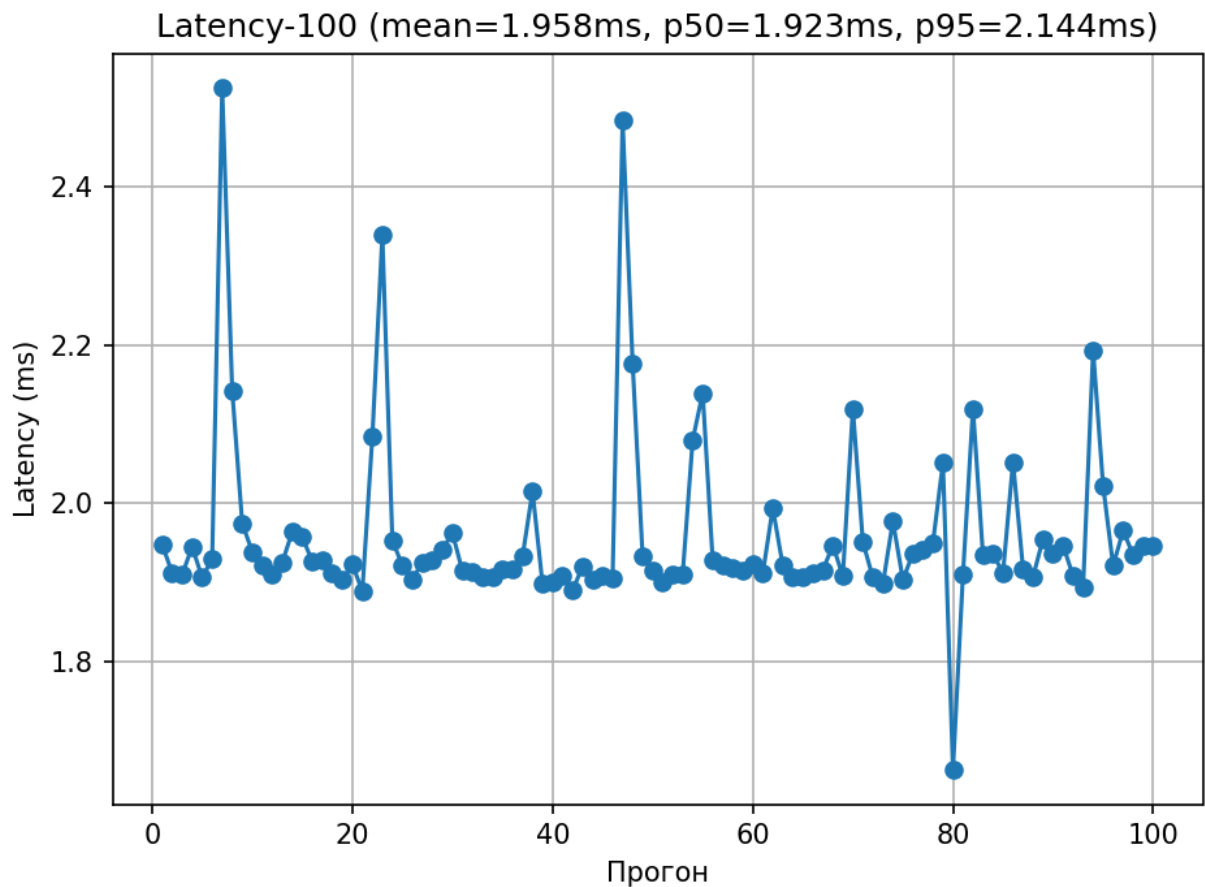


Рисунок 10 – график прогона одной картинки 100 раз

Латентность инференса была измерена для одной входной картинки (batch size = 1) на GPU.

Было выполнено 100 прогонов.

Средняя латентность составила 1.96 мс.

Полученные результаты свидетельствуют о стабильном и быстром инференсе модели.

## Задание 7 Seaborn-barplot

Подключения тензор  
import onnxruntime as ort

```
providers_trt = [
    "TensorrtExecutionProvider",
    "CUDAExecutionProvider",
    "CPUExecutionProvider"
]
```

Измерения throughput

```
session_trt = ort.InferenceSession(
    "model_cifar10.onnx",
    providers=providers_trt
)

def throughput(session, batch=256, steps=200):
    inp_name = session.get_inputs()[0].name
    x = np.random.rand(batch, 3, 32, 32).astype(np.float32)

    t0 = time.perf_counter()
    total = 0
    for _ in range(steps):
        session.run(None, {inp_name: x})
        total += batch
    t1 = time.perf_counter()

    return total / (t1 - t0)
```

```
(torch) C:\Users\Sanek\Desktop\Зимняя сессия 26год\Нейрокомпьютерные технологии\Lab3\lab\NC-lab4\src>python trt.py
onnxruntime: 1.23.2
Available providers: ['TensorrtExecutionProvider', 'CUDAExecutionProvider', 'CPUExecutionProvider']
[WARN] checkpoint 'checkpoint.tar' not found -> benchmarking random weights
[OK] Exported ONNX: model_cifar10.onnx
[CUDA] Throughput: 15929.0 images/sec (batch=256)
[TensorRT] Throughput: 25252.9 images/sec (batch=256)
Speedup (TRT vs CUDA): 1.59x
```

Рисунок 11 – Вывод Cuda vs TRT

Для ускорения инференса была использована библиотека ONNX Runtime с подключённым TensorRT Execution Provider.

Производительность измерялась в throughput (images/sec) при batch size = 256.

По сравнению с CUDA Execution Provider использование TensorRT позволило увеличить пропускную способность с 15 929 до 25 253 изображений в секунду, что соответствует ускорению примерно в 1.59 раза.

## Задание 8 INT8-точность

### Подключение

```
import onnxruntime as ort

# ONNX CUDA
sess_cuda = ort.InferenceSession(
    "model_cifar10.onnx",
    providers=["CUDAExecutionProvider", "CPUExecutionProvider"]
)

trt_opts = {"trt_int8_enable": "False", "trt_fp16_enable": "True"}
sess_trt = ort.InferenceSession(
    "model_cifar10.onnx",
    providers=["TensorrtExecutionProvider", "CUDAExecutionProvider",
               "CPUExecutionProvider"],
    provider_options=[trt_opts, {}, {}]
)

assert sess_trt.get_providers()[0] == "TensorrtExecutionProvider"
```

### Расчеты и плот

```
# pytorch_ips, onnx_ips, trt_ips получены через замер throughput (images/sec)
df = pd.DataFrame({
    "Backend": ["PyTorch", "ONNX (CUDA)", "TensorRT (FP16)"],
    "Speed-up": [1.0, onnx_ips / pytorch_ips, trt_ips / pytorch_ips]
})

sns.set_theme(style="whitegrid", font_scale=1.2)
plt.figure(figsize=(8, 5))
ax = sns.barplot(data=df, x="Backend", y="Speed-up")

for p in ax.patches:
    ax.annotate(f'{p.get_height():.2f}x',
                (p.get_x() + p.get_width()/2, p.get_height()),
                ha="center", va="bottom")

plt.title("Speed-up: PyTorch vs ONNX vs TensorRT (FP16)")
plt.ylabel("Speed-up (relative to PyTorch)")
plt.xlabel("Backend")
plt.tight_layout()

os.makedirs("docs", exist_ok=True)
plt.savefig("docs/speedup.png", dpi=150)
```

```

(torch) C:\Users\Sanev\Desktop\зимняя сессия 2016\д\нейрокомпьютерные технологии\lab3\lab\nc-lab4\src>python s
onnxruntime: 1.23.2
Available providers: ['TensorrtExecutionProvider', 'CUDAExecutionProvider', 'CPUExecutionProvider']
Device: cuda
[WARN] checkpoint 'checkpoint.tar' not found -> benchmarking random weights
[WARN] checkpoint 'checkpoint.tar' not found -> benchmarking random weights
[OK] Exported ONNX: model_cifar10.onnx
[PyTorch] Throughput: 16013.9 images/sec (batch=256)
[ONNX CUDA] Throughput: 16117.3 images/sec (batch=256)
TRT session providers: ['TensorrtExecutionProvider', 'CUDAExecutionProvider', 'CPUExecutionProvider']
[TensorRT FP16] Throughput: 82785.6 images/sec (batch=256)
Speedup (ONNX CUDA vs PyTorch): 1.01x
Speedup (TensorRT FP16 vs PyTorch): 5.17x
Saved: docs\speedup.png

```

Рисунок 12 – вывод сравнительной таблицы

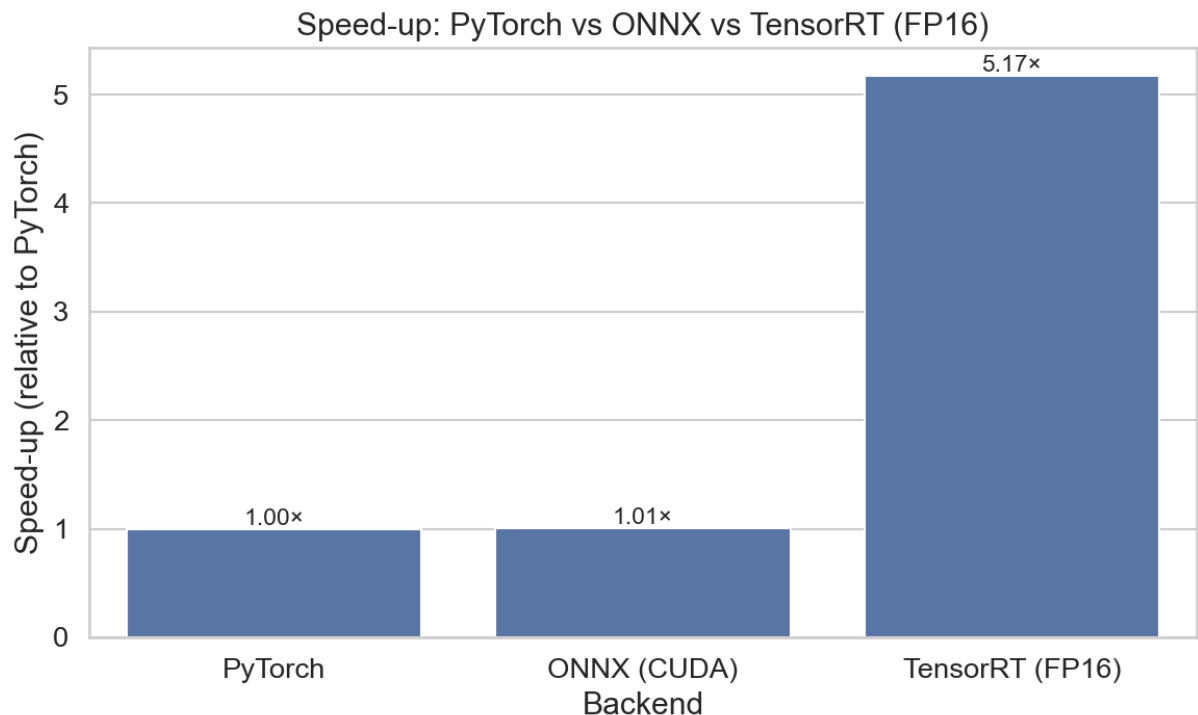


Рисунок 13 – сравнительная скорость

По замерам throughput при batch=256 получено: PyTorch = 16013 img/s, ONNX(CUDA) = 16117 img/s, TensorRT(FP16) = 82786 img/s.

### Задание 9

```

from torch.ao.quantization.quantize_fx import prepare_fx, convert_fx
from torch.ao.quantization import get_default_qconfig_mapping

```

# Настройка квантования

```

qconfig_mapping = get_default_qconfig_mapping("fbgemm")
example_inputs = (torch.randn(1, 3, 32, 32),)

```

# Подготовка модели к INT8

```

prepared = prepare_fx(fp32_model, qconfig_mapping, example_inputs)

```

# Калибровка на реальных данных

```

for x, _ in calib_loader:

```



```
prepared(x)
```

```
# Конвертация в INT8
int8_model = convert_fx(prepared)
```

```
ckpt = torch.load(path, map_location=device)
OK] Loaded checkpoint: checkpoint.tar
FP32] Top-1 on 1000 images: 83.70% (time=6.12s)
/Users/Sanek/miniconda3/envs/torch/Lib/site-packages/torch/ao/quantization/observer.py:229: UserWarning: Please use quant_min and quant_max to specify the range for observer
warnings.warn(
[INT8] Built INT8 model (time=5.54s)
[INT8] Top-1 on 1000 images: 83.60% (time=4.88s)
aved: docs/quantization.md
```

Рисунок 14 - Квантование

В результате эксперимента было выполнено сравнение Top-1 точности модели ResNet18 на датасете CIFAR-10 в формате float32 и после INT8-квантования.

Точность FP32 модели составила 83.70%, после INT8-квантования — 83.60%, что соответствует падению всего 0.10 п.п.

Полученные результаты показывают, что INT8-квантование позволяет существенно снизить разрядность вычислений с практически незаметной потерей качества, что делает данный подход перспективным для ускорения инференса на CPU.

## Теоретическая часть

### Аугментация данных

Аугментация данных представляет собой набор методов искусственного расширения обучающей выборки за счёт применения различных преобразований к исходным изображениям. К таким преобразованиям относятся повороты, отражения, масштабирование, сдвиги, изменения яркости и контрастности, а также добавление шума. Основная цель аугментации — повышение обобщающей способности модели и снижение переобучения.

В данной работе для аугментации использовалась библиотека **Albumentations**, которая предоставляет широкий набор высокоэффективных и гибко настраиваемых трансформаций изображений. Применение аугментаций позволяет модели видеть больше разнообразных вариантов входных данных, что особенно важно при обучении глубоких нейронных сетей на ограниченных датасетах, таких как CIFAR-10.

Использование аугментации приводит к более устойчивому обучению и, как правило, к повышению точности модели на тестовой выборке.

### Сверточные нейронные сети и EfficientNet

Сверточные нейронные сети (CNN) являются основным инструментом для решения задач компьютерного зрения. Они используют свёрточные слои для извлечения пространственных признаков изображений, что позволяет эффективно обрабатывать данные с локальной структурой.

Архитектура **EfficientNet** основана на принципе сбалансированного масштабирования глубины, ширины и разрешения входных данных сети. В отличие от традиционных подходов, где масштабируется только один

параметр, EfficientNet использует составной коэффициент масштабирования, что позволяет достичь лучшего соотношения между точностью и вычислительной сложностью.

В работе проводилось сравнение моделей EfficientNet-B0 и EfficientNet-V3. Модель B0 является более компактной и быстрой, тогда как V3 обладает большей вычислительной сложностью и потенциально более высокой точностью. Экспериментально было показано, что при ограниченном числе эпох и данных более лёгкая модель может демонстрировать лучшие результаты за счёт более стабильного обучения.

### **Тензоры и вычисления на GPU**

Тензор является базовой структурой данных в библиотеках глубокого обучения, таких как PyTorch. Тензоры обобщают понятия скаляров, векторов и матриц и позволяют эффективно выполнять параллельные вычисления.

Использование графических процессоров (GPU) значительно ускоряет операции над тензорами благодаря массовому параллелизму. В данной работе вычисления выполнялись как на CPU, так и на GPU с использованием CUDA, что позволило сравнить производительность различных подходов к инференсу моделей.

### **ONNX и ONNX Runtime**

ONNX (Open Neural Network Exchange) — это открытый формат для представления моделей машинного обучения, обеспечивающий переносимость моделей между различными фреймворками и средами выполнения.

ONNX Runtime представляет собой высокопроизводительный движок инференса, поддерживающий различные вычислительные провайдеры, включая CPU, CUDA и TensorRT. Использование ONNX Runtime позволяет ускорить инференс моделей по сравнению с нативным запуском в PyTorch, особенно в продакшн-сценариях.

В ходе работы модель была экспортирована из PyTorch в формат ONNX, после чего выполнялись измерения пропускной способности и латентности инференса.

### **TensorRT**

TensorRT — это высокопроизводительная библиотека от NVIDIA, предназначенная для оптимизации и ускорения инференса нейронных сетей на GPU. TensorRT выполняет такие оптимизации, как слияние слоёв, оптимизация графа вычислений и использование пониженной точности (FP16 и INT8).

### **Квантование и INT8-точность**

Квантование представляет собой метод снижения разрядности представления весов и активаций нейронной сети, например, с float32 до int8.

Это позволяет уменьшить объём памяти модели и ускорить вычисления, особенно на CPU.

В работе использовалось посттренировочное квантование (Post-Training Quantization)

|                      |                |       |     |
|----------------------|----------------|-------|-----|
| DZ/#9-отчет          | now            | 0   0 | ... |
| DZ/#8-int8-точность  | 7 minutes ago  | 0   4 | ... |
| DZ/#7-Speed          | 41 minutes ago | 0   3 | ... |
| DZ/#6-TensorRT       | 1 hour ago     | 0   2 | ... |
| DZ/#5-latency        | 2 hours ago    | 0   1 | ... |
| DZ/#4-Early_stopping | 2 hours ago    | 0   1 | ... |
| DZ/#2-альбомента-ауг | 3 hours ago    | 0   1 | ... |
| DZ/#1----gpu-отчет   | 4 hours ago    | 0   2 | ... |

Бранчи